

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA51

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO5: *Develop the network security strategies based on the study of viruses, worms, firewall architectures, and IDS/IPS functionalities.CO (BL5 , BL6)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A

PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Code of Conduct:

*I **KESHAVARAM L/192511253** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

L. Keshavam

Annexure A

PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

1. Design pseudocode for a **stateful firewall system** that tracks active connections and filters packets based on session state. Evaluate how state tracking improves security compared to stateless filtering.

2. Design pseudocode for a **signature-based malware detection system** that scans files and compares them against a known signature database. Evaluate its effectiveness in detecting known threats and its limitations against new attacks.

3. Design pseudocode for a **distributed denial-of-service (DDoS) detection system** using traffic threshold analysis. Evaluate how the algorithm differentiates between legitimate high traffic and attack traffic.

4. Design pseudocode for a **network intrusion detection system using machine learning classification**. Evaluate how feature selection and training data impact detection accuracy.

5. Design pseudocode for a **secure network access control system** that authenticates users and enforces role-based access policies. Evaluate how access control mechanisms enhance overall network security.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Question 1 – Stateful Firewall System

Question

Design pseudocode for a stateful firewall system that tracks active connections and filters packets based on session state. Evaluate how state tracking improves security compared to stateless filtering.

Answer

1. Problem Understanding

A stateful firewall maintains information about active communication sessions. Instead of examining each packet independently, the firewall checks whether a packet belongs to a valid established connection, starts a new permitted session, or represents an invalid or unexpected packet.

2. Inputs, Outputs, and Constraints

Item	Description
Input	Incoming packet containing source IP, destination IP, source/destination port, protocol, and session flags.
State table	Active connection records containing session information and current state.
Policy rules	Rules defining permitted new connections and restricted traffic.
Output	ALLOW, DROP, or LOG action.
Constraint	State table must be updated consistently and expired sessions must be removed.

3. Algorithm

1. Receive an incoming packet.
2. Extract packet header and session information.
3. Search the state table for a matching connection.

4. If the connection is established and the packet is valid for the current state, allow it and update the session.
5. If no session exists, check whether the packet is permitted to start a new connection.
6. If permitted, create a new state-table entry and allow the packet.
7. Otherwise, drop and log the packet.
8. Remove expired sessions periodically.

4. Pseudocode

ALGORITHM StatefulFirewall(Packet)

INPUT:

Packet = {srcIP, dstIP, srcPort, dstPort, protocol, flags}
StateTable
FirewallRules

OUTPUT:

ALLOW, DROP, or LOG

BEGIN

session ← FindSession(StateTable, Packet)

IF session EXISTS THEN

IF ValidStateTransition(session, Packet) THEN

UpdateSession(session, Packet)

RETURN ALLOW

ELSE

Log("Invalid state transition")

RETURN DROP

END IF

ELSE

IF IsNewConnectionAllowed(Packet, FirewallRules) THEN

newSession ← CreateSession(Packet)

Add(StateTable, newSession)

```
        RETURN ALLOW
    ELSE
        Log("New connection denied")
        RETURN DROP
    END IF
END IF
END
```

5. Supporting Functions

```
FUNCTION FindSession(StateTable, Packet)
    FOR each session IN StateTable
        IF SessionMatches(session, Packet) THEN
            RETURN session
        END IF
    END FOR
    RETURN NOT_FOUND
END FUNCTION

FUNCTION ValidStateTransition(session, Packet)
    IF Packet.flags are valid for session.state THEN
        RETURN TRUE
    ELSE
        RETURN FALSE
    END IF
END FUNCTION

FUNCTION RemoveExpiredSessions(StateTable)
    FOR each session IN StateTable
        IF CurrentTime > session.expiryTime THEN
            Remove(session)
        END IF
    END FOR
END FUNCTION
```

6. Evaluation: Stateful vs Stateless Filtering

Aspect	Stateful Firewall	Stateless Firewall
Connection awareness	Tracks active sessions.	Examines packets independently.
Security context	Uses session state.	Uses packet header rules only.
Unexpected packets	Can reject invalid session transitions.	May require explicit rules for each case.
Complexity	Higher because state must be maintained.	Simpler.
Resource usage	Requires state-table memory.	Generally lower state memory.
Security advantage	Better context-aware filtering.	Less context-aware.

Final Answer

A stateful firewall should maintain an active connection table and validate packets against the current session state. State tracking improves security because packets are evaluated in the context of legitimate connections, allowing unexpected or invalid session traffic to be rejected more effectively than purely stateless filtering.

Question 2 – Signature-Based Malware Detection System

Question

Design pseudocode for a signature-based malware detection system that scans files and compares them against a known signature database. Evaluate its effectiveness in detecting known threats and its limitations against new attacks.

Answer

1. Problem Understanding

Signature-based detection identifies malware by comparing file characteristics, such as hashes or known patterns, with a database of known malicious signatures. It is efficient for previously identified threats but cannot reliably identify malware for which no matching signature exists.

2. Inputs and Outputs

Item	Description
Input	File or directory selected for scanning.
Signature database	Known malicious file hashes or patterns.
Output	CLEAN, MALWARE DETECTED, or SCAN ERROR.
Constraint	Signature database should be updated regularly.

3. Pseudocode

ALGORITHM SignatureMalwareScan(File)

INPUT:

File

SignatureDatabase

OUTPUT:

CLEAN, MALWARE_DETECTED, or ERROR

BEGIN

IF File does not exist THEN

RETURN ERROR

END IF


```

IF File is inaccessible THEN
    RETURN ERROR
END IF

fileHash ← CalculateHash(File)

IF fileHash IN SignatureDatabase THEN
    Log("Known malware detected")
    Quarantine(File)
    RETURN MALWARE_DETECTED
END IF

patterns ← ExtractKnownPatterns(File)

FOR each signature IN SignatureDatabase DO
    IF Matches(signature, patterns) THEN
        Log("Known malware signature matched")
        Quarantine(File)
        RETURN MALWARE_DETECTED
    END IF
END FOR

RETURN CLEAN
END

```

4. Signature Database Update

```

ALGORITHM UpdateSignatures(NewSignatures)

BEGIN
    FOR each signature IN NewSignatures DO
        IF signature is valid THEN
            AddOrUpdate(SignatureDatabase, signature)
        END IF
    END FOR

```

```
Log("Signature database updated")  
END
```

5. Evaluation

Factor	Evaluation
Known threats	Highly effective when the malware matches a current signature.
Speed	Generally fast because comparison can use hashes and predefined patterns.
Unknown malware	Weak if no signature exists.
Zero-day attacks	May not detect them until a signature becomes available.
Database dependency	Detection quality depends on timely signature updates.

6. Example

If a scanned file has a SHA-256 hash that appears in the known-malware database, the algorithm identifies the file as malicious and quarantines it. If the file is a newly created malware sample with no matching signature, the signature-based algorithm may classify it as clean.

Final Answer

Signature-based detection is highly effective and efficient for known malware when the signature database is current. Its major limitation is that new, modified, or zero-day malware without a matching signature may evade detection, so it is best complemented by behavioral or anomaly-based detection.

Question 3 – DDoS Detection Using Traffic Threshold Analysis

Question

Design pseudocode for a distributed denial-of-service (DDoS) detection system using traffic threshold analysis. Evaluate how the algorithm differentiates between legitimate high traffic and attack traffic.

Answer

1. Problem Understanding

A threshold-based DDoS detector monitors traffic volume and related characteristics over a defined time window. Traffic that significantly exceeds an established baseline may indicate an attack. However, legitimate events such as product launches or scheduled campaigns can also create high traffic, so volume alone should not automatically trigger blocking.

2. Inputs and Outputs

Item	Description
Input	Packet/flow counts, source information, destination information, time window, and baseline traffic.
Threshold	Maximum expected traffic level or deviation from baseline.
Output	NORMAL, SUSPICIOUS, or DDOS_ALERT.
Constraint	Use baseline comparison and persistence checks to reduce false alarms.

3. Pseudocode

ALGORITHM DetectDDoS(TrafficWindow)

INPUT:

TrafficWindow

BaselineRate
ThresholdMultiplier
PersistenceLimit

OUTPUT:

NORMAL, SUSPICIOUS, or DDOS_ALERT

BEGIN

currentRate $\leftarrow$ CalculateTrafficRate(TrafficWindow)
deviation $\leftarrow$ currentRate / BaselineRate

IF deviation $\leq$ ThresholdMultiplier THEN
 suspiciousCount $\leftarrow$ 0
 RETURN NORMAL
END IF

IF IsKnownLegitimateEvent(TrafficWindow) THEN
 suspiciousCount $\leftarrow$ 0
 Log("High traffic classified as legitimate")
 RETURN NORMAL
END IF

suspiciousCount $\leftarrow$ suspiciousCount + 1

IF suspiciousCount $\geq$ PersistenceLimit THEN
 RETURN DDOS_ALERT
ELSE
 RETURN SUSPICIOUS
END IF

END

4. Distributed Monitoring

ALGORITHM DistributedDDoSMonitor(Sensors)

BEGIN

```

FOR each sensor IN Sensors DO
    localResult ← DetectDDoS(sensor.TrafficWindow)
    SendToCentralAnalyzer(sensor.ID, localResult)
END FOR

combinedScore ← AggregateSensorResults()

IF combinedScore >= GlobalAttackThreshold THEN
    TriggerMitigation()
    GenerateAlert()
END IF
END

```

5. Legitimate High Traffic vs Attack Traffic

Indicator	Legitimate High Traffic	Possible DDoS Traffic
Traffic volume	High but expected.	Sudden and unusually high.
Duration	May follow a known schedule.	May persist unexpectedly.
Sources	Often normal/known user distribution.	May show abnormal source patterns.
Application behavior	Valid requests and successful sessions.	Excessive, malformed, or repetitive requests may appear.
Event context	Can correlate with planned business activity.	No legitimate business explanation.

6. Trade-Off

A low threshold detects attacks quickly but can cause false positives during legitimate traffic spikes. A high threshold reduces false positives but may delay detection. Using a baseline, persistence requirement, source/application indicators, and distributed sensor correlation provides a better balance.

Final Answer

Use traffic thresholds together with baseline comparison, persistence checks, legitimate-event validation, and distributed sensor correlation. This avoids treating every traffic spike as an attack and improves the ability to distinguish genuine high demand from sustained abnormal traffic.

Question 4 – Network Intrusion Detection Using Machine Learning Classification**Question**

Design pseudocode for a network intrusion detection system using machine learning classification. Evaluate how feature selection and training data impact detection accuracy.

Answer**1. Problem Understanding**

A machine-learning-based IDS classifies network observations as normal or malicious based on features extracted from traffic. The quality of the selected features and training data strongly affects accuracy, false positives, and the ability to generalize to new traffic.

2. Inputs and Outputs

Item	Description
Training data	Labeled network observations containing normal and attack examples.
Features	Selected traffic characteristics used by the classifier.
Model	Trained classification algorithm.
New traffic	Network observation to classify.
Output	NORMAL or ATTACK, optionally with confidence score.

3. Training Pseudocode

ALGORITHM TrainIDSMModel(Dataset)

INPUT:

Labeled Dataset

OUTPUT:

Trained Model

BEGIN

CleanDataset(Dataset)

RemoveDuplicateRecords(Dataset)

Features $\leftarrow$ ExtractFeatures(Dataset)

SelectedFeatures $\leftarrow$ SelectRelevantFeatures(Features)

TrainSet, TestSet $\leftarrow$ SplitDataset(Dataset)

Model $\leftarrow$ TrainClassifier(TrainSet, SelectedFeatures)

Predictions $\leftarrow$ Model.Predict(TestSet)

Metrics $\leftarrow$ Evaluate(Predictions, TestSet.labels)

IF Metrics meet required thresholds THEN

Save(Model)

RETURN Model

ELSE

AdjustFeaturesOrTrainingData()

REPEAT training process

END IF

END

4. Classification Pseudocode

ALGORITHM DetectIntrusion(NetworkFlow, Model)

INPUT:

NetworkFlow

Trained Model

OUTPUT:

NORMAL or ATTACK

BEGIN

features $\leftarrow$ ExtractFeatures(NetworkFlow)

selected $\leftarrow$ ApplySelectedFeatures(features)

result, confidence $\leftarrow$ Model.Predict(selected)

IF result = ATTACK AND confidence $\geq$ DetectionThreshold THEN

Log("Intrusion detected")

GenerateAlert(NetworkFlow)

RETURN ATTACK

ELSE

RETURN NORMAL

END IF

END

5. Feature Selection Impact

- Relevant features can improve classification quality and reduce unnecessary processing.
- Irrelevant features can introduce noise and reduce generalization.
- Too few features may remove important attack characteristics.
- Feature selection can reduce model complexity and inference cost.

6. Training Data Impact

Training Data Issue	Effect
---------------------	--------

Insufficient attack examples	Poor detection of underrepresented attacks.
Class imbalance	Model may favor the majority class.
Outdated data	Performance may decline against newer traffic patterns.
Noisy or mislabeled data	Can reduce classification reliability.
Unrepresentative data	Model may perform poorly in the real network.

7. Evaluation Metrics

Metric	Purpose
Accuracy	Overall proportion of correct classifications.
Precision	Proportion of predicted attacks that are actually attacks.
Recall	Proportion of actual attacks correctly detected.
False-positive rate	Measures benign traffic incorrectly classified as malicious.

Final Answer

A machine-learning IDS should extract relevant features, train on representative and well-labeled data, evaluate the model, and use a confidence threshold during detection. Feature selection reduces noise and computational cost, while balanced and current training data improves generalization and detection accuracy.

Question 5 – Secure Network Access Control with Role-Based Policies

Question

Design pseudocode for a secure network access control system that authenticates users and enforces role-based access policies. Evaluate how access control mechanisms enhance overall network security.

Answer

1. Problem Understanding

A secure access-control system first verifies a user's identity and then determines what resources the authenticated user is allowed to access. Role-Based Access Control (RBAC) assigns permissions to roles such as Administrator, Employee, or Guest rather than assigning every permission individually to every user.

2. Inputs and Outputs

Item	Description
Input	Username, authentication credential, requested resource, and requested action.
User database	Identity and assigned role information.
Role policy	Permissions associated with each role.
Output	ACCESS_GRANTED or ACCESS_DENIED.
Security controls	Authentication, authorization, logging, session timeout, and account protection.

3. Authentication and Authorization Pseudocode

ALGORITHM SecureNetworkAccess(User, Credential, Resource, Action)

INPUT:

User

Credential

Resource
Action

OUTPUT:

ACCESS_GRANTED or ACCESS_DENIED

BEGIN

IF User does not exist THEN

 Log("Unknown user")

 RETURN ACCESS_DENIED

END IF

IF AccountIsLocked(User) THEN

 Log("Locked account attempted access")

 RETURN ACCESS_DENIED

END IF

IF NOT Authenticate(User, Credential) THEN

 RecordFailedAttempt(User)

 IF FailedAttempts(User) >= MaxAttempts THEN

 LockAccount(User)

 END IF

 Log("Authentication failed")

 RETURN ACCESS_DENIED

END IF

role ← GetUserRole(User)

IF IsSessionExpired(User) THEN

 Log("Session expired")

 RETURN ACCESS_DENIED

END IF

```

permission ← GetPermission(role, Resource, Action)

IF permission = ALLOW THEN
    Log("Access granted")
    RETURN ACCESS_GRANTED
ELSE
    Log("Access denied")
    RETURN ACCESS_DENIED
END IF
END

```

4. Role Management

```

FUNCTION GetPermission(role, Resource, Action)

BEGIN
    permissions ← RolePermissions[role]

    IF (Resource, Action) EXISTS IN permissions THEN
        RETURN ALLOW
    ELSE
        RETURN DENY
    END IF
END FUNCTION

```

5. Example RBAC Policy

Role	Example Permissions
Administrator	Manage users, security settings, and approved network resources.
Employee	Access assigned business applications and approved internal resources.
Guest	Access only explicitly permitted guest services.

6. Security Benefits

- Authentication prevents unidentified users from accessing protected resources.
- Authorization ensures users receive only the permissions assigned to their role.
- Least privilege reduces the impact of compromised accounts.
- Account lockout and failed-attempt tracking reduce credential-guessing risk.
- Logging provides an audit trail for successful and failed access attempts.
- Session controls reduce risk from abandoned or expired sessions.

7. Trade-Offs

Strong authentication and detailed logging improve security but add administrative and processing overhead. RBAC simplifies permission management compared with individual user permissions, but roles must be reviewed regularly to avoid excessive privileges.

Final Answer

Use a layered access-control algorithm that authenticates the user, checks account status, retrieves the user's role, verifies the requested resource and action against role permissions, and records the result. RBAC and least-privilege policies strengthen network security by ensuring that authenticated users receive only the access required for their role.